

Introductions to Big Data Systems: Homework 11

Francisco Diego Ariel Romero Contreras

法兰西思科

francisco.romero.c@ug.uchile.cl

2025403471

Spark Streaming top-k

The implement of this algorithm use a spark context that take the streaming words and use a map reduce over it to count the words, once the streaming stops the program create a RDD where the result is sorted and select only the top-100 most common words.

As the sorting and select top process are neither associative nor accumulative operation, this part of the algorithm must be outside of the streaming aggregation process.

To run this program should be used two shells, in shell 1 run `./generator.sh` on the generator folder, in shell 2 run `./submit.sh` on the `simple_app_python` folder.

To finish the counting must be use `ctrl + c` over the word streaming process and the counting words process, for the counting job there is a timeout of ten minutes but it can also be manually shutdown.

The homework has 3 folders and the report, the code path is `simple_app_python/SimpleApp.py`, the output is in a folder with each batch process result

hw11

```
|— generator
|— hw11.md
|— hw11output
|   |— part-1765722420000
|   |   |— part-00000
|   |   |— part-00001
|   |   |— _SUCCESS
|   |— part-1765722480000
|   |   |— part-00000
|   |   |— part-00001
|   |   |— _SUCCESS
|   |— part-1765722540000
|   |   |— part-00000
|   |   |— part-00001
|   |   |— _SUCCESS
|   |— part-1765722600000
|   |   |— part-00000
|   |   |— part-00001
|   |   |— _SUCCESS
|   |— part-1765722660000
|   |   |— part-00000
|   |   |— part-00001
|   |   |— _SUCCESS
|   |— .....
|   |— .....
|   |— .....
|— simple_app_python
```

simple_app_python/SimpleApp.py:

```
from __future__ import print_function
import sys
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

sc = SparkContext(appName="Py_HDFSWordCount")
ssc = StreamingContext(sc, 60)

lines = ssc.textFileStream("hdfs://intro00:8020/user/2025403471/stream")
counts = lines.flatMap(lambda line: line.split(" ")).map(lambda x: (x,
1)).reduceByKey(lambda a, b: a+b)

def top_100(rdd):
    return sc.parallelize(
        rdd.takeOrdered(100, key=lambda x: -x[1])
    )

top_counts = counts.transform(top_100)

top_counts.pprint()
top_counts.saveAsTextFiles("hdfs://intro00:8020/user/2025403471/hw11output/part")

ssc.start()
ssc.awaitTerminationOrTimeout(600) # 10 minutes max
```